Title: Legal Document Assistant - Agentic AI Capstone

Name: SAMBODHITA BHOI

Roll Number: 23053542

Batch/Program: Agentic AI Course (Course Code: IS30056)

Date: April 21, 2026

Live Application - *https://bettercallsamb.streamlit.app/*

Problem Statement

Reading, analyzing, and extracting specific clauses from large volumes of case documents, contracts, and non-disclosure agreements is heavily time-consuming for paralegals and junior lawyers. Legal professionals require immediate, precise answers regarding specific contractual obligations without the risk of an AI hallucinating binding legal advice or fabricating facts not present in the text.

The objective of this project is to build a highly reliable, RAG-enabled Legal Document Assistant. This system must answer user queries based strictly on a dynamically loaded knowledge base of 50 legal documents. Crucially, the system must recognize when it does not know the answer, refuse to invent information, and possess the ability to step outside the legal knowledge base using specialized tools to define general legal terminology.

Solution and Features

To solve this problem, I developed an Agentic AI system utilizing a LangGraph StateGraph architecture. The solution acts as an intelligent intermediary between the legal professional and the case files. The core features of this system include:

- **Strict RAG-Grounded Retrieval:** The system retrieves chunks of legal text from a ChromaDB vector store and restricts the LLM to answer using only that context.
- **Multi-Tool Routing:** A master routing node determines the intent of the user's question. Queries about specific contracts are routed to the retrieval database, while requests for general legal definitions (e.g., "What is a good faith clause?") are routed to a live DuckDuckGo web search tool.
- **Self-Reflection and Quality Gating:** The agent evaluates its own generated answers against the retrieved text. If the answer is deemed unfaithful or hallucinated, the graph triggers a retry mechanism to correct the response before presenting it to the user.
- **Persistent Contextual Memory:** Utilizing LangGraph's MemorySaver, the agent tracks

conversation history via a thread_id, allowing the paralegal to ask follow-up questions naturally without restating previous context.

Tech Stack

The project was built entirely in Python using a modular architecture, leveraging the following modern AI frameworks and libraries:

- **Orchestration Framework:** LangGraph (StateGraph, MemorySaver)
- **Large Language Model (LLM):** Groq API
- **Embeddings Model:** SentenceTransformers (all-MiniLM-L6-v2)
- **Vector Database:** ChromaDB (In-memory ephemeral storage)
- **External Tooling:** DuckDuckGo Search API (ddgs)
- **Frontend User Interface:** Streamlit

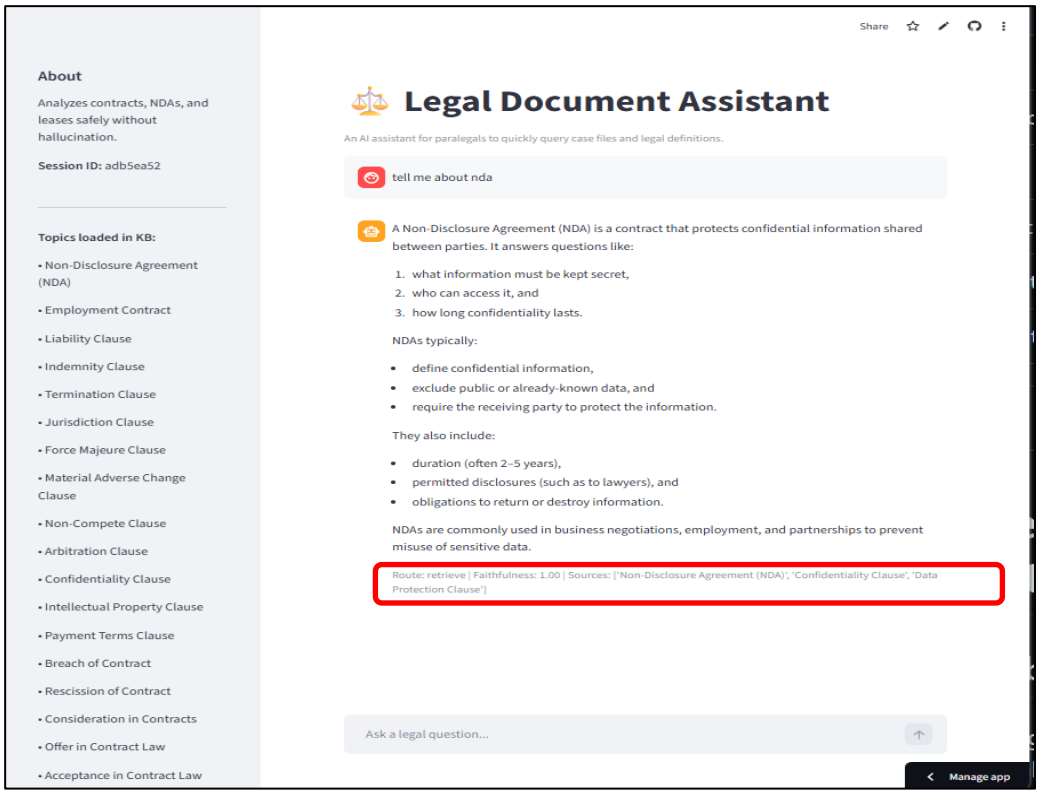
System Architecture and Workflow

The system's logic is cleanly separated from the routing mechanism via a customized State dictionary (TypedDict). The workflow follows these distinct nodes:

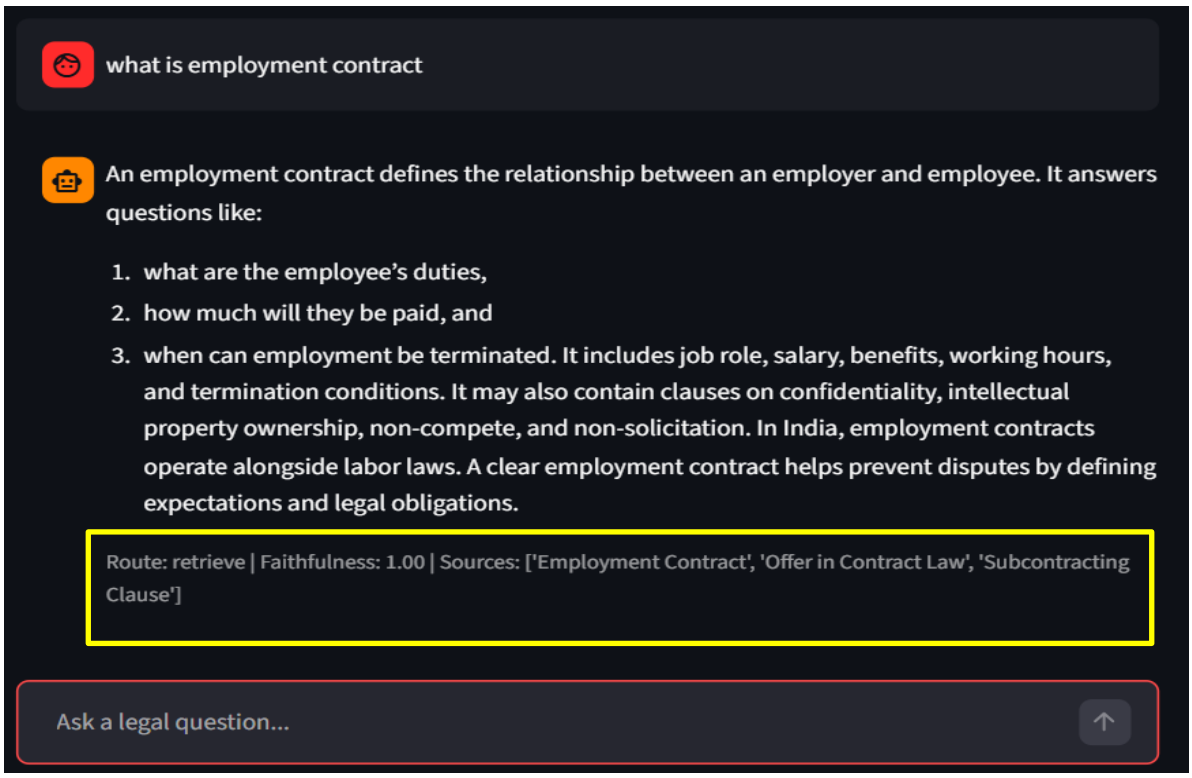
- **Memory Node:** Ingests the user query, appends it to the history, and applies a sliding window of the last 6 messages to preserve token limits while maintaining context.
- **Router Node:** Analyzes the query and past context to make a definitive choice: route to retrieve, tool, or memory_only.
- **Retrieval Node:** Embeds the user query and fetches the top 3 most relevant legal clauses from the ChromaDB collection.
- **Tool Node:** Executes a live web search if the user requires a general legal definition not contained in the case files.
- **Answer Node:** Compiles the system prompt, enforcing strict grounding rules, and synthesizes the final response.
- **Evaluation Node (Self-Reflection):** Acts as a quality gate. It scores the faithfulness of the answer against the retrieved context (0.0 or 1.0). If it fails, the edge loops back to the Answer Node for a rewrite, capped at two retries to prevent infinite loops.

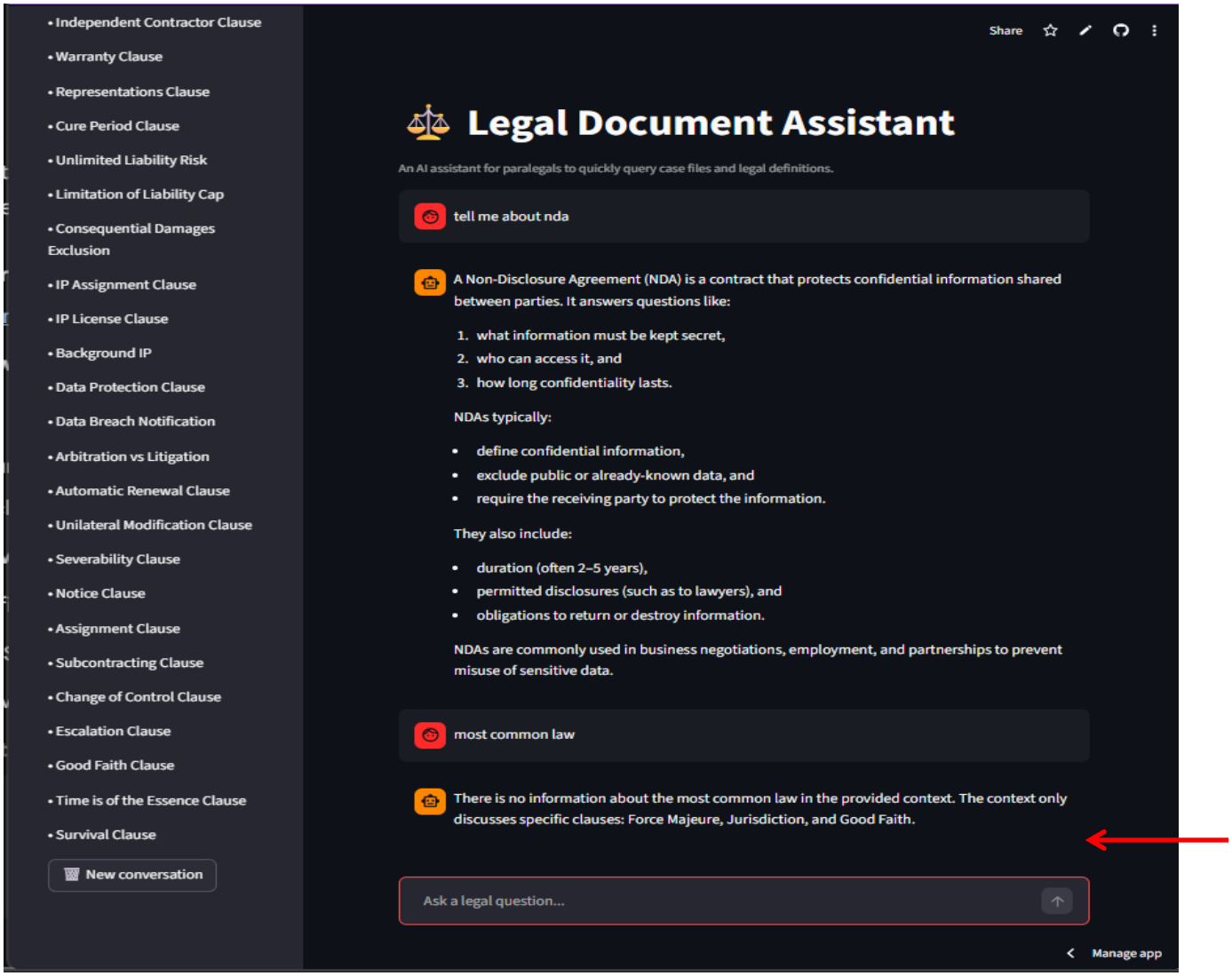
Deployment and Implementation

Live Application - *https://bettercallsamb.streamlit.app/*

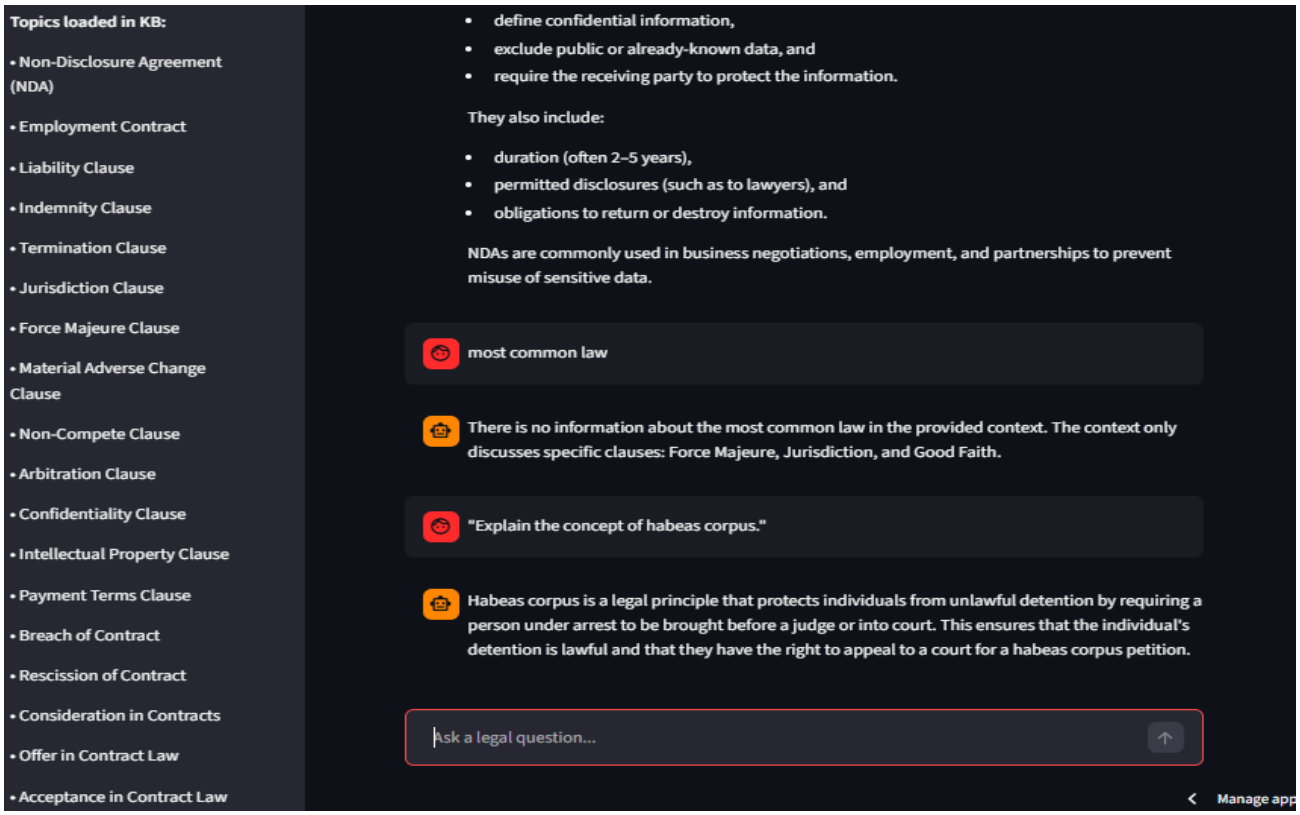


Demonstration of the agent correctly routing a query to the retrieval node, extracting the correct legal document, and returning an accurate answer.





Demonstration of the agent's evaluation node working correctly, preventing hallucination when the requested information is absent from the knowledge base.



The agent successfully routing a general definition query to the DuckDuckGo web search tool.

(Habeas corpus not in the knowledge base)

Project Insights & Future Scope

Unique Points

This project stands out due to several advanced engineering decisions that go beyond a standard RAG pipeline:

- **Dynamic JSON Knowledge Base:** Rather than hardcoding a few documents, the system dynamically ingests a .json file containing 50 distinct legal clauses, complete with metadata (Titles and Categories) which are actively passed to the LLM for better contextual understanding.
- **Zero-Hallucination Tolerance:** In the legal domain, hallucination is unacceptable. The eval_node strictly forces the model to verify its own factual accuracy. If the text does not contain the answer, the agent is structurally forced to admit it does not have the information rather than guessing.
- **Resilient Tool Design:** External APIs often fail. The tool_node is wrapped in exception handling that catches HTTP timeouts or search errors and returns them as strings, ensuring the graph architecture never crashes during runtime.

Future Improvements

Given more time and resources, the system could be improved in the following technical areas:

- **Advanced Document Parsing (OCR):** Currently, the system ingests clean JSON text. Integrating a library like pdfplumber or Tesseract OCR would allow paralegals to upload scanned PDF case files directly into the Streamlit UI, which the system would chunk and embed on the fly.
- **Hybrid Search Retrieval:** ChromaDB currently uses dense vector search (semantic similarity). Upgrading the retrieval node to use a Hybrid Search approach—combining dense embeddings with BM25 keyword search—would drastically improve retrieval accuracy for highly specific legal contract numbers or exact statutes.
- **Streaming Callbacks:** The current Streamlit UI waits for the entire LangGraph execution to finish before printing the answer. Implementing asynchronous streaming callbacks would allow the LLM's response to appear word-by-word, significantly improving the perceived latency and user experience.

Conclusion

The development of this Legal Document Assistant successfully demonstrates the practical value of Agentic AI in highly specialized, risk-averse domains. By combining a strict LangGraph-driven RAG architecture with self-reflective evaluation and dynamic tool routing, this system bridges the gap between static case files and immediate, actionable insights. It provides paralegals and junior lawyers with a secure, hallucination-free assistant that significantly reduces the time spent on manual document review while safely handling out-of-scope definitions via web search. Ultimately, this capstone project proves that with rigorous engineering safeguards, generative AI can be a highly reliable and transformative tool in daily legal operations.